

Research paper

CarNet: A generative convolutional neural network-based line-of-sight/non-line-of-sight classifier for global navigation satellite systems by transforming multivariate time-series data into images

Ni Zhu^a, Ruiwen He^b, Zhiqiang Wang^b

^a AME-GEOLOC, University Gustave Eiffel, F-44344 Bouguenais, France

^b Léonard de Vinci Pôle Universitaire, Research Center, 92916, Paris La Défense, France

ARTICLE INFO

Keywords:

Global navigation satellite systems
Line-of-sight
Non-line-of-sight
Deep learning
Generative convolutional neural network
Positioning

ABSTRACT

Urban environments present significant challenges to commercial Global Navigation Satellite Systems (GNSS) receivers due to degraded satellite visibility and Non-line-of-sight (NLOS) receptions. Mitigating NLOS receptions for GNSS is essential, especially for safety-critical and reliability-critical location-based applications. Traditional physical error channel propagation modeling encountered bottlenecks since the NLOS and multipath errors cannot be modeled accurately in complex urban environments. Data-driven methods show significant potential for effectively classifying GNSS Line-of-sight (LOS) and NLOS. This paper proposes the CarNet - a generative Convolutional Neural Network (CNN)-based GNSS LOS/NLOS classifier by transforming multivariate time-series data into images. CarNet comprises two modules: an image generator and an image classifier. The image generator enriches and augments the original 1-dimension feature vector into 2-dimension feature maps and the image classifier uses an inception-based CNN to realize multi-scale feature extraction and classification. The proposed architecture is trained and tested on more than 6 h of real vehicle data collected in different challenging environments (about 1.6 million samples). A thorough benchmark is conducted, comparing CarNet against the existing mainstream Artificial Intelligence (AI) methods. The results with cross-validation on unseen data indicate that CarNet achieves the highest accuracy, i.e., 81.47% while maintaining the optimal balance between precision for both classes: 83.3% for LOS and 70.99% for NLOS. Finally, positioning accuracy is assessed using a reweighting strategy based on the LOS/NLOS information predicted by CarNet. The assessment of total datasets shows that CarNet weighting can achieve the best accuracy compared to the traditional weighting schemes based on signal-to-noise ratio or satellite elevation. CarNet shows strong potential for embedding into GNSS receivers to enhance positioning accuracy in complex urban environments, benefiting a wide range of location-based applications such as autonomous driving, emergency response, and urban logistics.

1. Introduction

With an increasing number of location-based applications embedded in connected devices, Global Navigation Satellite Systems (GNSS) play a more and more important role in everyone's daily life. On the one hand, a wide range of GNSS-assisted location-based applications has emerged recently in all the market sectors. These applications are no longer limited to entertainment utilities but have already extended to safety-critical and reliability-critical applications. For example, some location-based smart canes exist to better guide visually impaired people while facilitating their daily mobility; some smart-connected accessories can

help parents track their children in case of danger; a new concept of insurance "Pay-as-you-drive" calculates the users' invoices based on their traveled locations; in industry 4.0, automatic logistic robots and asset tracking systems can significantly enhance efficiency, reduce human error, and streamline operations based on high accurate positions.

On the other hand, GNSS suffers from signal degradation in challenging urban environments. This is because GNSS receivers use the trilateration principle to determine users' positions, measuring the pseudorange from each visible satellite to the user based on Time of Arrival (TOA), as illustrated in Fig. 1(a). When satellite signals are reflected by the surrounding obstacles, i.e., NLOS reception (as shown

* Corresponding author.

E-mail address: ni.zhu@univ-eiffel.fr (N. Zhu).

<https://doi.org/10.1016/j.engappai.2025.110160>

Received 10 May 2024; Received in revised form 18 December 2024; Accepted 23 January 2025

Available online 4 February 2025

0952-1976/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

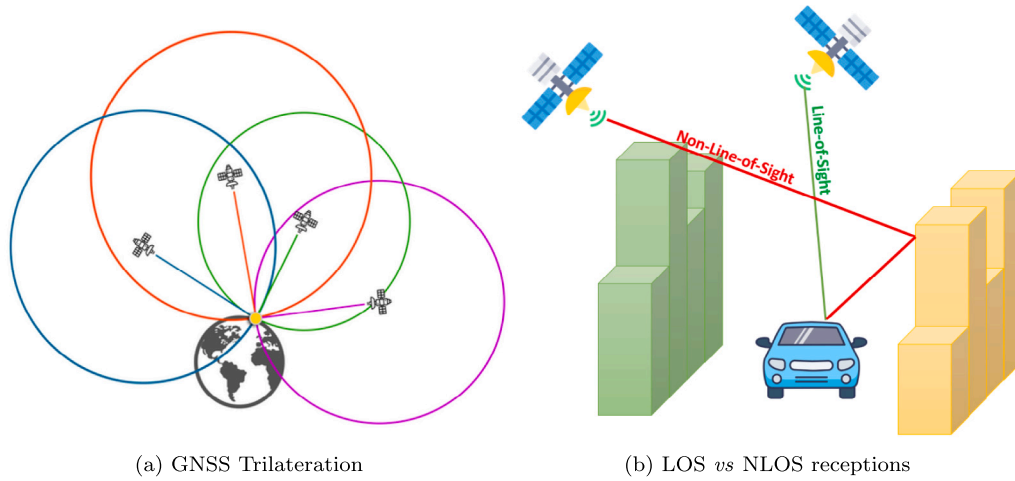


Fig. 1. GNSS positioning principle and LOS/NLOS reception.

in Fig. 1(b)), huge errors will be introduced to the range measurements leading to inaccurate position estimation.

The motivation behind this research is to develop a robust AI-based classifier capable of accurately distinguishing between GNSS line-of-sight (LOS) and non-line-of-sight (NLOS) signal receptions across a wide range of challenging urban environments. Effectively mitigating or deweighting NLOS signals in the position estimation algorithm can improve GNSS accuracy, thereby enhancing the performance of various safety-critical and reliability-critical location-based applications, such as autonomous driving, emergency response, and transportation systems in densely populated or obstructed areas.

Many hardware-based GNSS NLOS mitigation techniques exist such as dual polarization antennas (Groves et al., 2010), antenna arrays (Vagle et al., 2016), or beamformings (Fohlmeister et al., 2017), etc. By evaluating the difference of the carrier-power-to-noise-density (C/N_0) measurements from left-hand circular polarized outputs subtracted from the right-hand circular polarized C/N_0 counterpart, the dual-polarization antenna can detect NLOS reception (Jiang and Groves, 2014). The beamforming technique focuses the antenna reception in specific directions and rejects signals from undesirable directions. However, hardware-based solutions may struggle to adapt to fast-changing environments caused by high dynamic motions. Moreover, they are often cumbersome and costly with high power consumption, which presents huge constraints for wearable devices. That is why, the majority of the NLOS detection techniques focused on software design from a signal processing perspective.

Most of the existing traditional GNSS NLOS mitigation algorithms are realized through signal propagation channel modeling and measurement consistency checks using residual redundancy (Zhu et al., 2017) or other robust estimators (Wang et al., 2023a). The most well-known algorithms, such as Receiver Autonomous Integrity Monitoring (RAIM) and Advanced RAIM (ARAIM) (Blanch et al., 2012), are widely used in the aviation domain. However, these classic statistic-based fault detection methods operate under the assumption of Gaussian-distributed measurement errors and typically handle only one fault at a time. With the growth of global urbanization, an increasing number of complex modern architectures as well as new construction materials appear such as vegetation walls, reflective facades, solar panels, etc. These challenging urban configurations create significant difficulties for model-based NLOS mitigation techniques. They lead to non-Gaussian error distributions and frequent occurrence of faulty measurements, violating the assumptions in traditional statistical fault detection approaches. The NLOS error profile is too difficult to be accurately characterized by one universal physical model since they can vary depending on location, time as well as surrounding environments.

The data-driven approach strives to build models using extensive data for phenomena that are challenging to explicitly model with laws of physics or mathematics. The applications of AI have led to substantial advancements in failure detection and fault analysis across various industrial sectors. For example, Xu et al. (2023) proposed a cross-modal fusion Convolutional Neural Network (CNN) to diagnose mechanical faults. Li et al. (2023) developed a multibranch denoising CNN for intelligent fault diagnosis of rolling bearings under noisy conditions. Wang et al. (2023b) designed a knowledge extraction system using Variational Auto-encoder for failure analysis of semiconductor components. Song et al. (2024) proposed an anti-disturbance state estimation method for reaction-diffusion neural networks (RDNNs) under persistent dwell-time (PDT) switching, which could benefit industries that rely on complex networked systems.

Similarly, many AI-based methods have emerged for GNSS measurement Fault Detection and Exclusion (FDE), particularly NLOS mitigation. This is because AI-based learning methods enable the avoidance of constrained assumptions present in statistic-based FDE methods, which transformed the way to address navigation integrity problems. Many learning-based algorithms are already employed in the current literature to distinguish GNSS LOS/NLOS, ranging from the basic ones like Random Forest, Support Vector Machine (SVM) (Suzuki and Amano, 2021), Decision Tree, K-Nearest Neighbors (Zhang et al., 2018), Gradient Boosting (Sun et al., 2020), to advanced deep learning techniques such as CNN (Suzuki et al., 2020), Long Short-Term Memory (LSTM) (Zhang et al., 2021). The features used are typically at the raw measurement or correlator level. The primary challenges and limitations of the existing methods are their generalizability to new scenarios and their sensitivity to feature selection.

To overcome the previously mentioned limitations, this paper proposes a Generative CNN (GCNN)-based GNSS LOS/NLOS classifier by transforming multivariate time-series data into images. The global architecture of the proposed algorithm is illustrated in Fig. 2. The main contributions of this paper are as follows:

- Propose a methodology of converting time-series data into images using a lightweight convolutional image generator, which enables feature augmentation to enhance higher-level hierarchical feature extractions;
- Design an inception-based CNN architecture to classify the generated images that improves significantly the stability and capacity of generalization of the network, showing strong potential for real-time applications, even when running on a CPU;
- Make a systematic benchmark of the proposed method against the existing mainstream ML methods (all the tree-based methods,

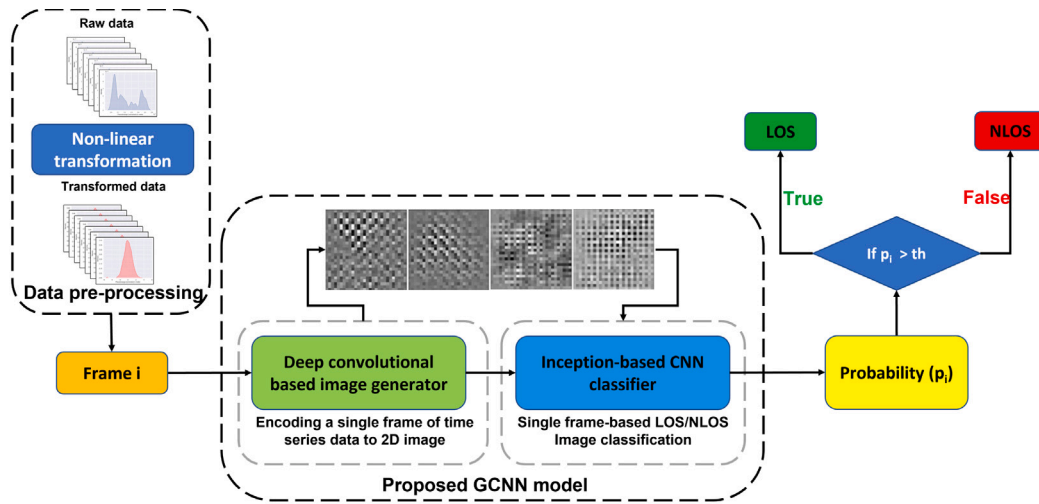


Fig. 2. An overview of the proposed LOS/NLOS detection system.

Table 1
GNSS LOS/NLOS labeling technique summary.

Labeling techniques	Description	Labeling error sources	Accuracy Level	Related Research work
3DMA	(1) Ray-tracing: If the ray between the reference position and the satellite has an intersection with one building, the reception is labeled as NLOS; (2) Sky mask: Construct skyplot with building boundaries, then project the satellite on it.	Reference solution error, Map data error, satellite ephemeris error	++++	Hsu (2017), Miura et al. (2015), Ng et al. (2019), Betaille et al. (2013)
Fisheye camera	If the projected satellite is in the sky zone of the fisheye image, it is labeled as NLOS	Reference solution error, Camera orientation estimation error, Image segmentation error, satellite ephemeris error	+++	Suzuki and Amano (2021), Marais et al. (2020), Zhu et al. (2023b)
Residual error thresholding	Comparing the residual error between the recalculated pseudorange via a reference trajectory and the measured one.	Reference solution error, satellite ephemeris error, pseudorange reconstruction errors, threshold definition error	++	Luo et al. (2023)

LSTM, MLP, Multi-head Attention) for GNSS LOS/NLOS classification using a huge real dataset collected in challenging urban environments.

The remainder of the paper is structured as follows: Section 2 summarizes the state-of-the-art regarding AI-based GNSS LOS/NLOS classification techniques as well as the existing approaches for converting time series into images. Section 3 presents the details of the dataset, the labeling technique, and the candidate features. The proposed methodology and the architecture of CarNet are described in Section 4. Section 5 provides an ablation study on model architecture as well as the classification performance evaluation benchmarking with mainstream AI techniques. Positioning accuracy using the information provided by CarNet is also evaluated in the same section. Section 6 draws conclusions and proposes future research perspectives.

2. State of the art

2.1. AI-based GNSS LOS/NLOS labeling and classification

In recent years, more and more research work emerged using data-driven approaches to detect GNSS NLOS receptions. For supervised Machine learning (ML) methods, LOS/NLOS labeling is a great challenge. Among the existing research work, the LOS/NLOS true labels are obtained in general by the following approaches as summarized in Table 1:

– 3D Map Assisted (3DMA) techniques: these methods can be implemented in the following ways.

- * Ray-tracing: It labels the GNSS LOS/NLOS signals by tracing the path between the reference position and the satellite. If the traced signal path has an intersection with one building, the reception is labeled as NLOS; otherwise, it is labeled as LOS (Hsu, 2017; Miura et al., 2015).
- * Sky mask: It constructs a sky mask by projecting building boundaries onto the skyplot. Satellites projected within the sky zone are labeled as LOS, while those projected onto buildings are labeled as NLOS (Betaille et al., 2013; Ng et al., 2019).

These geometry-based approaches are susceptible to errors from reference position inaccuracy, map data as well as satellite ephemeris errors which impact satellite position accuracy.

- Fisheye camera: This method uses a fisheye camera mounted on the receiver to capture the sky view from the receiver's perspective. Following image segmentation and frame transformation, tracked satellites are projected onto the image; those within the sky area are labeled as LOS, while others are labeled as NLOS (Suzuki and Amano, 2021; Marais et al., 2020; Zhu et al., 2023b). This technique can be affected by reference position errors, frame transformation inaccuracies due to incorrect camera orientation estimation, image segmentation errors, and satellite ephemeris inaccuracies.

- Pseudorange residual error thresholding: This method recalculates the pseudorange using the reference position, satellite position as well as the existing signal propagation error models accounting for factors like satellite clock error, ionospheric and tropospheric delay, and signal-to-signal bias on the measured original pseudorange. The pseudorange residual is deduced by calculating the difference between the measured pseudorange and the reconstructed one. If the residual exceeds a set threshold, the signal is labeled as NLOS (Luo et al., 2023). Error sources involved in this method include reference solution errors, satellite ephemeris errors, pseudorange reconstruction errors, and threshold definition errors. Moreover, Additionally, fixed thresholds may lack generalizability across scenarios and devices.

Once the data is labeled, different features can be selected and decorrelated before entering into the training stage. In the current literature, the candidate features for GNSS LOS/NLOS classification include raw measurements (e.g., pseudorange, carrier phase, Doppler, Signal-to-Noise Ratio, satellite elevation) as well as some statistical ones (differenced raw measurements, variance, etc.). Different ML models are studied to distinguish GNSS signal LOS/NLOS receptions including SVM (Suzuki and Amano, 2021), Xu et al. (2020), Hsu (2017)), Decision Tree (Yozevitch et al., 2016), K-Nearest Neighbors (Zhang et al., 2018), Gradient boosting (Sun et al., 2020), Light Gradient Boosting Machine (LGBM) (Luo et al., 2023), etc. The classification accuracy of the LOS/NLOS varies between 70%–95% in the existing literature but they are not comparable since it depends on the test data size and scenario. For example, Suzuki and Amano (2021) has obtained a 97.7% accuracy for NLOS detection while the tests are conducted on five static points. Not many research works use deep learning techniques for example Neural Networks are used in Suzuki and Amano (2021), Suzuki et al. (2020) while LSTM is applied in Zhang et al. (2021). The main limitations of the existing AI-based GNSS LOS/NLOS classification methods lie in their high sensitivity in terms of feature choice as well as their lack of generalization capacity. The models usually need to be trained in the same scenarios with similar dynamics.

2.2. Encoding time-series data into images: existing techniques and our motivations

A range of existing techniques have been proposed in the state-of-the-art to convert Univariate Time Series (UTS) to images for AI-based classification and imputation. Common techniques include Gramian Angular Field (GAF) (Wang et al., 2015), Markov Transition Fields (MTF) (Wang et al., 2015), Recurrence Plot (RP) (Eckmann et al., 1995), Short Time Fourier Transform (STFT), Continuous Wavelet Transformation (CWT), Hilbert Huang Transform (HHT) (Huang et al., 1998) as well as pixel replication and interpolation (Sayed et al., 2023). These techniques can be applied directly to UTS, whereas fusion is needed when converting Multivariate Time Series (MTS) into 2D images, which presents more challenges. One of the most popular methods for MTS imaging is RGB or multi-spectral channel fusion which assigns different variables to different color channels (Chen and Wang, 2022).

Imaging time series for AI model training has been applied in different domains, such as IoT security, energy management, smart manufacturing, and healthcare, which are proven to have superior performance compared to traditional ML algorithms based on the extraction of statistical features. Abdel-Basset et al. (2020) designed a streamlined architecture for Human Activity Recognition (HAR) by transforming the time series data into images using RP, MTP, and GAF encoding, addressing the challenges posed by the heterogeneous nature of sensory data. For the authentication of the IoT devices, Baldini et al. (2019) proposed an approach to convert the Radio Frequency (RF) emissions from 11 devices to images based on RP. The accuracy result has shown significant improvement compared to the traditional feature

Table 2

An overview of dataset.

Tracks	Environments	Number of observations	Duration	Percentage of LOS (%)
Nantes center	DU ^a	42 560	11 min 5 s	79.83
Full Nantes	DU, SU ^b	166 033	59 min 40 s	70.31
Paris XII	DU	357 762	1 h 51 min 53 s	72.51
Boulevards	HDU	710 400	4 h 27 min 10 s	69.86
La Défense	HDU ^c	166 998	49 min 15 s	79.22
Toulouse	DU	117 576	45 min 55 s	46.56
Total	SU, DU, HDU	1 561 329	9 h 4 min 58 s	70.04

^a DU: Deep Urban.

^b SU: Suburban.

^c HDU: Highly Deep Urban (with skyscrapers).

extraction method, i.e., from 91.3% to 96.8%. Sharma et al. (2019) proposed the methodology DeepInsight, which constructs an image by placing similar elements or features together and dissimilar ones further apart, enabling the collective use of neighboring elements. The performance evaluations on a gene-expression dataset, a text dataset, and a vowel dataset demonstrate favorable outcomes.

In our context, the GNSS signal patterns exhibit significant variability, especially in stringent urban environments. Lower-level hierarchical features might not accurately convey the reception states. They also have the potential to introduce instability to AI models. The existing ML-based GNSS LOS/NLOS classifiers suffer from high sensitivity on feature selection and low generability, which depend strongly on selected features and the training scenario. We propose in this paper a methodology based on Generative CNN to convert time-series data into images and then train an inception-based classifier. The motivations are as follows:

- Traditional ML-based LOS/NLOS classifiers, such as the tree-based methods, use different GNSS features but they can hardly explore higher spatial or temporal hierarchical features. By converting time series into images, some hidden patterns can be better exposed and extracted by leveraging CNN-related techniques.
- Compared to the existing converting techniques, GCNN can provide the effect of data augmentation on the existing features so that the sensitivity on feature selection is reduced and the model stability is enhanced.
- Transforming time series data into images can unlock the potential of CNN as well as the use of GPU in the training stage.
- Converting the GNSS MTS into images can pave the way for several promising perspectives: such as facilitating multimodal data training (map data, semantic information, etc.) as well as leveraging transfer learning for other transport modality using some powerful pre-trained models such as ResNet (He et al., 2016) Inception series (Szegedy et al., 2016, 2017), etc.

3. Dataset, data labeling and candidate features

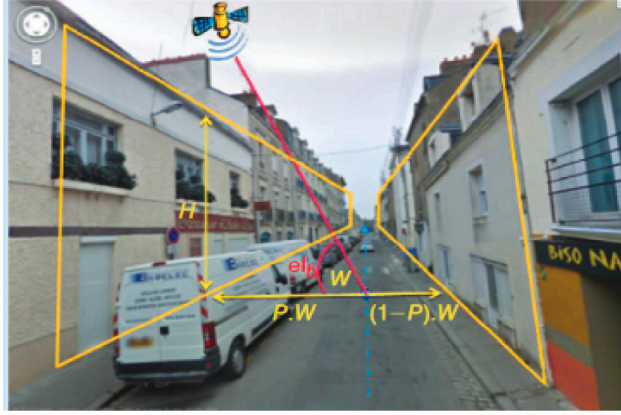
3.1. Dataset and its labeling

The dataset utilized in this paper was collected at 5 Hz using a low-cost GNSS receiver (Ublox LEO 6T) installed on the roof of the Vehicle for Experimental Research on Trajectories (VERT) of the GEOLOC Laboratory as shown in Fig. 3(a). The traveled environments include suburban, deep urban in Nantes and Toulouse, France as well as the highly deep urban area with skyscrapers in Paris. The dataset includes six tracks with in total 1561329 MTS observations over more than 9 h. Table 2 summarizes the details of the dataset.

The data is binarily labeled as LOS/NLOS using the ray-tracing technique based on the Urban Multipath Model proposed in Zhu (2018), which is an improved version of the Urban Trench Model in Betaille et al. (2013) as shown in Fig. 3(b).



(a) Instrumental vehicle VERT



(b) Ray-tracing labeling (Betaille et al. 2013)

Fig. 3. Data collection and labeling.

3.2. Candidate features

In order to facilitate the benchmark with mainstream ML techniques and to further delve into the potential of the proposed method, we extract or generate the following 24 candidate features from the GNSS receiver for further selection.

3.2.1. Low hierarchical features

Low hierarchical features include Code pseudorange, Carrier phase, Doppler, Signal-to-Noise Ratio (SNR), Satellite elevation, Azimuth, and Number of satellites at each epoch. Figs. 4 and 5 present the distributions of SNR and Satellite elevation for LOS and NLOS signals across two datasets. LOS signals generally have higher SNR values and satellite elevations, whose peaks are around 45–50 dB and 65–75 degrees respectively, indicating better signal quality. On the contrary, NLOS signals dominate the lower SNR and satellite elevation ranges, with peaks respectively around 30–35 dB and 15–25 degrees, indicating poorer signal quality. There are overlapping regions with intermediate signal quality, where distinguishing between LOS and NLOS based solely on one feature becomes challenging for the model.

3.2.2. High hierarchical features

- The first derivative of the carrier phase $\dot{\phi}$:

$$\dot{\phi} = \frac{\phi_i - \phi_{i-1}}{\Delta t} \quad (1)$$

where the ϕ_i is the carrier phase at the epoch i and Δt is the time interval between two consecutive epochs.

- Pseudorange residual Pr calculated with the Weighted Least Squares (WLS) method using a C/N_0 weighting model as described in Zhu et al. (2017). Fig. 6 plots two examples of pseudorange residual distributions for LOS and NLOS signals. The

distributions of the LOS pseudorange residual are centered around zero with sharp peaks, which indicate higher accuracy and better signal quality. The pseudorange residuals of NLOS exhibit a right-skewed behavior with long positive tails.

- Epoch-wisely Normalized Pseudorange Residual (NPR) (Ozeki and Kubo, 2022):

$$NPR = \frac{Pr_i - Pr_{min}}{Pr_{max} - Pr_{min}} \quad (2)$$

where, Pr_i represents the pseudorange residual of the satellite i , Pr_{max} and Pr_{min} refer respectively to the maximum and the minimum pseudorange residuals at the current epoch. Fig. 7 shows distributions of NPRs from two datasets. The overall distributions have huge overlap zones between LOS and NLOS. From zoomed-in views, we can observe that the NPRs of LOS signals are centered towards lower values while NLOS NPRs are more spread out.

- Code-Minus-Carrier (CMC): the difference between the pseudorange and the carrier phase range in meters.
- Carrier Phase Range Rate Consistency (CPRRC), which is the absolute difference of the changing rate of the carrier phase deduced from the carrier phase measurement and from Doppler:

$$CPRRC = |\Delta\rho_{\phi,i} - \Delta\rho_{D,i}| \quad (3)$$

where $\Delta\rho_{\phi,i} = \lambda\phi_i - \lambda\phi_{i-1}$ represents the carrier phase range changing rate calculated from carrier phase measurements of two consecutive epochs with λ the wavelength; $\Delta\rho_{D,i} = -\lambda f_{D,i}\Delta t$ represents the phase range rate calculated from Doppler measurement $f_{D,i}$ at the epoch i .

- Pseudorange Rate Consistency PRC :

$$PRC = \Delta P_i - \Delta\rho_{D,i} \quad (4)$$

where, $\Delta P_i = P_i - P_{i-1}$ represents the changing rate of the pseudorange deduced from the pseudorange measurements of two consecutive epochs; $\Delta\rho_{D,i} = -\lambda f_{D,i}\Delta t$ is the changing rate of the pseudorange calculated from the Doppler shift as mentioned above.

- Test statistic of the Global Test (GT) in the traditional integrity monitoring, i.e. Normalized Sum Square Error (NSSE) and that of the Local Test (LT), i.e., normalized residual using WLS C/N_0 solver and weighting as described in Zhu et al. (2020).
- Sigma C/N_0 using Danish reweighting method (Leick et al., 2015) with (C/N_0) weighting model (Zhu et al., 2020):

$$\sigma_{i,j+1}^2 = \sigma_{i,0}^2 \begin{cases} e^{\frac{w_{i,j}}{th}} & \text{if } w_{i,j} > th \\ 1 & \text{if } w_{i,j} < th \end{cases} \quad (5)$$

where, $\sigma_{i,j+1}^2$ is the variance of the i th measurement error at $(j+1)$ th iteration; $\sigma_{i,0}^2$ is the a priori variance of the i th measurement error with the C/N_0 weighting model; $w_{i,j}$ is the local test statistic of the i th measurement at j th iteration, th is the local test threshold.

- Pseudorange residual using Danish reweighting method;

3.2.3. Rolling features

- SNR Fluctuation Magnitude (SFM) (Tokura and Kubo, 2017):

$$SFM(i) = \sqrt{\frac{1}{N} \sum_{n=i-N}^i (SNR(t_{i-n}) - SNR_{mean})^2} \quad (6)$$

where N represents the window size, $SNR(t_{i-n})$ is the SNR at the $(i-n)$ th epoch and SNR_{mean} is the mean SNR in the sliding window. Considering the vehicular dynamic, the window size here is fixed as 5, which represents a duration of 1 s. The same window size is applied for the following rolling features.

- Variance of pseudorange, variance of pseudorange residual and variance of SNR: $Var(P)$, $Var(Pr)$, $Var(SNR)$;

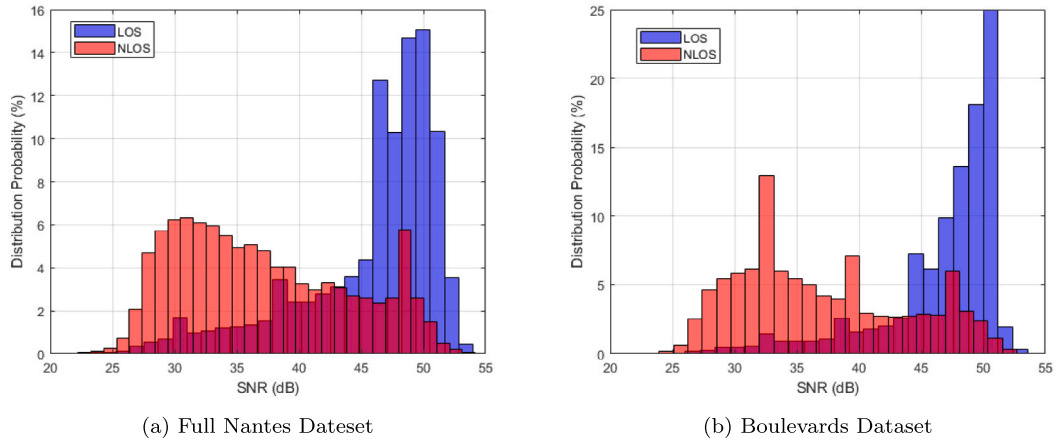


Fig. 4. Distribution of SNR for LOS and NLOS in two different cities.

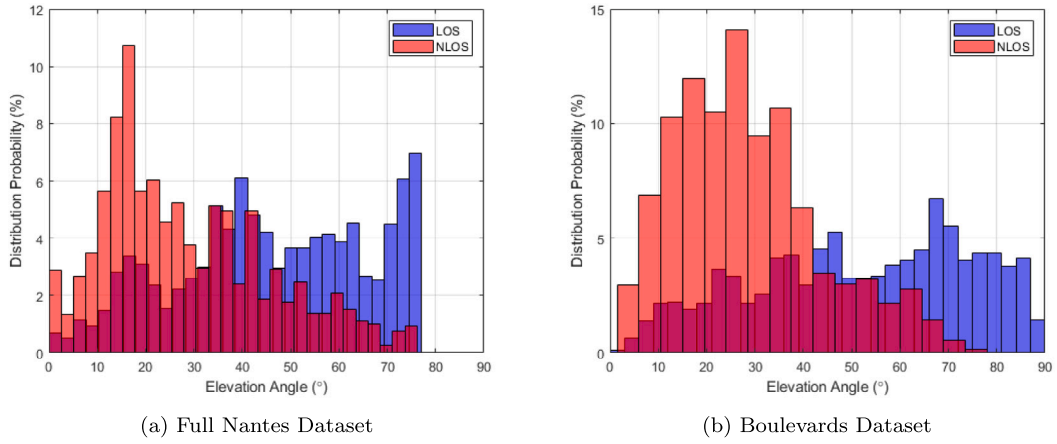


Fig. 5. Distribution of satellite elevation for LOS and NLOS in two different cities.

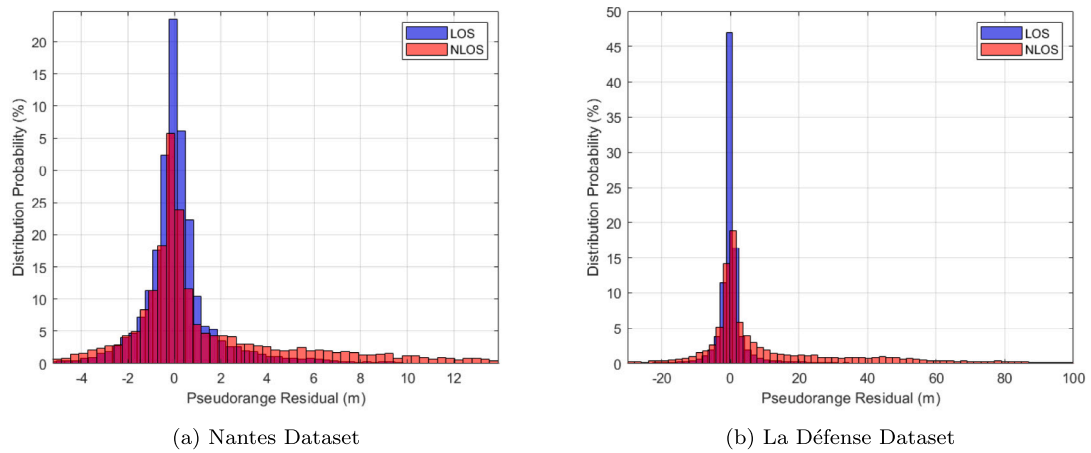


Fig. 6. Distribution of pseudorange residual for LOS and NLOS in two different cities.

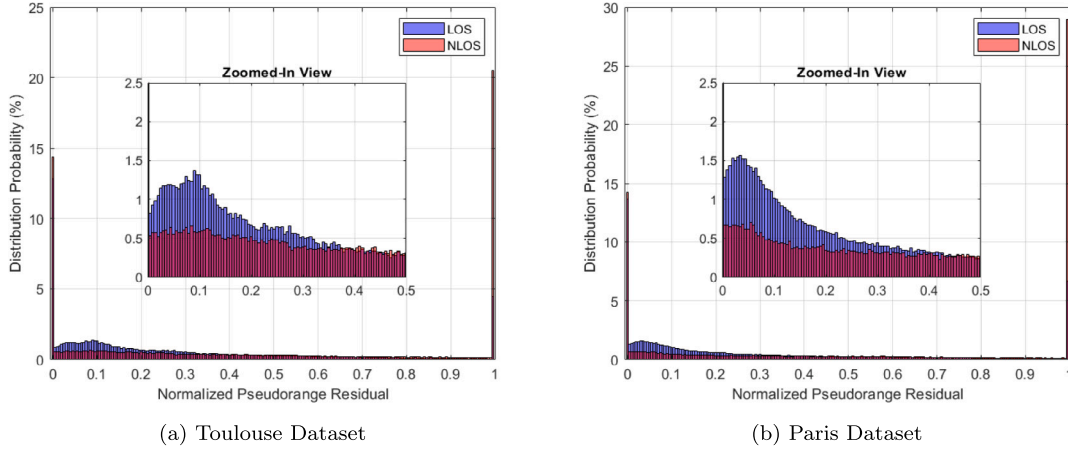


Fig. 7. Distribution of Normalized Pseudorange Residual for LOS and NLOS in two different cities.

- Entropy of pseudorange, entropy of pseudorange residual and entropy of SNR: $E(P)$, $E(Pr)$, $E(SNR)$.

In our previous research work (Zhu et al., 2023a), 10 core features are selected using different ML-based feature engineering techniques as well as expert knowledge: SNR , ϕ , NPR , CMC , $CPRRC$, LT , number of satellite, $Var(SNR)$, $Var(P)$, $Var(Pr)$.

4. CarNet: Proposed GNSS LOS/NLOS Classifier

4.1. Methodology description

Convolutional Neural Networks (CNN) have been gaining exciting results in the field of computer vision due to their powerful feature extraction capabilities and robustness. In this context, the use of CNN techniques for non-image data analysis has become widespread over multiple domains (Yang et al., 2019, Sayed et al., 2023), which have shown higher performance. Transforming non-image data analysis problems into computer vision tasks enables us to better leverage the potential of CNN.

This paper aims to construct a framework for classifying GNSS LOS/NLOS data using deep learning techniques by transforming time-series data into images. We propose a lightweight generative CNN (GCNN), in contrast to other methods, where the image generation process and the classification process are performed simultaneously. Algorithm 1 summarizes the major steps of our method. First, a non-linear transformation is applied to map the raw data to a Gaussian distribution. Then, the transformed 1D data is fed directly into our GCNN model through an image generator and a classifier. The image generator transforms the input feature into a 2D image by a trainable image encoder consisting of an MLP layer and two transposed convolutional layers. Finally, a multi-scale feature extraction CNN network is processed to recognize features in the image for classification. The overall architecture of the image generator and classifier is illustrated in Fig. 8. The details of the three major steps mentioned above are presented in the following subsections.

4.1.1. Non-linear transformation

For many ML and DL algorithms, the data is suggested to be rescaled or normalized into a small specific interval. This makes different features comparable and ensures the model learns patterns based on the intrinsic relationship of the data rather than the values of absolute magnitudes. It allows transforming the raw data into dimensionless pure values, facilitating the ability to compare and weight metrics with different units or magnitudes, and speeding up the convergence of the training network.

Algorithm 1 Generative CNN-based LOS/NLOS Classifier

```

1: Input: Dataset  $D$ 
2: Output: LOS/NLOS Prediction (LNP)
3: // Step 1: Feature Transformation for Robustness
4:  $D' \leftarrow \text{QuantileTransformer}(D)$   $\triangleright$  Normalizes data and prepares it
   for image generation
5: for each sample  $x \in D'$  do
6:   // Step 2: Image Generation for Enhanced Feature Representa-
   tion
7:    $\text{Image\_generated} \leftarrow \text{ImageGenerator}(x)$   $\triangleright$  Convert
   multi-dimensional time series features into 2D images
8:   // Step 3: LOS/NLOS Classification with Inception-based CNN
9:    $p \leftarrow \text{CNNClassifier}(\text{Image\_generated})$   $\triangleright$  Returns  $p \in (0, 1)$ ,
   indicating likelihood of NLOS
10:  // Step 4: LOS/NLOS Decision Threshold
11:  if  $p < 0.5$  then
12:     $LNP \leftarrow 0$   $\triangleright$  LOS - line-of-sight
13:  else
14:     $LNP \leftarrow 1$   $\triangleright$  NLOS - non-line-of-sight
15:  end if
16: end for

```

Typically, *Min-Max scaling* is often used for data normalization, i.e., mapping the data uniformly onto the interval $[0, 1]$. However, this approach is very sensitive to outliers, which may cause too much information loss (Rousseeuw and Leroy, 2005). In particular, when the data distribution is not uniform, normalization may lead to changes in data density, affecting the effectiveness of the algorithm (Box and Cox, 1964). Another widely used data transformation method is *Z-score normalization* (also known as *standardization*), which rescales features to have a mean of zero and a standard deviation of one. This method assumes the data follows a Gaussian distribution and is particularly useful when the underlying distribution of the features is normal or approximately normal (Bishop and Nasrabadi, 2006). However, like Min-Max scaling, Z-score normalization is also sensitive to outliers, as extreme values can disproportionately affect the mean and standard deviation.

Considering these limitations and the distribution of our data, we proposed to use a non-linear transformation, namely *Quantile Transformer* (QT), to preprocess our data. In machine learning, QT is used to transform features into a target distribution such as normal distribution or uniform distribution (Izenman, 2008). For a given feature, this transformation tends to disperse the most frequent values while reducing the impact of outliers (Friedman et al., 2009). In particular,

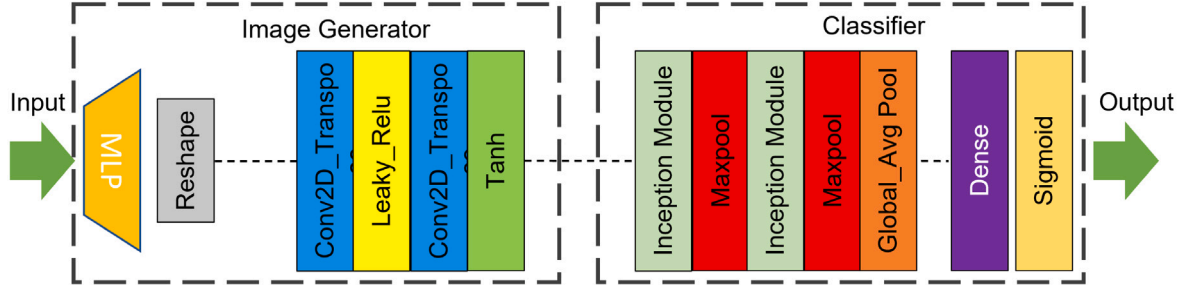


Fig. 8. Architecture of the proposed GCNN model, which consists of two modules: Image Generator and Image Classifier.

it can better handle variables of different scales and make the mostly asymmetrical distributed real-world observed variables more treatable to fulfill the basic underlying assumption of normality inherent to most statistical models (Tiku, 1988). For our analysis, we specifically opted to transform the data into a normal distribution, as this aligns with the assumptions of many statistical models and enhances the model's performance.

QT works by assigning a quantile to the ranked original data point, then mapping them to the target distribution using its inverse Cumulative Distribution Function (CDF). Specifically, for each feature $X = [x_1, x_2, \dots, x_n]$, the quantile rank r_i for each data point x_i can be written as:

$$r_i = \frac{\text{rank}(x_i)}{n + 1} \quad (7)$$

where, $\text{rank}(x_i)$ is the rank of x_i in the sorted list, n is the number of data points. If the target distribution is Gaussian (normal), the transformed value is obtained by applying the inverse CDF of the normal distribution $\Phi^{-1}(\cdot)$, as follows:

$$x_i^{\text{transformed}} = \Phi^{-1}(r_i) \quad (8)$$

QT is particularly useful when the data has a highly skewed distribution or contains outliers, as it spreads out the most frequent values and pushes extreme values towards the tails, reducing their impact (Wasserman, 2013). Using QT to achieve a normal distribution allows the model to leverage the properties of Gaussianity, such as well-defined mean and variance, thereby improving performance on tasks that assume normality in the underlying data distribution.

4.1.2. Image generator: Time-series data to 2D images

Several methods have been proposed for converting 1D time-series data into 2D image data, such as longitudinal data replication and resizing using linear interpolation (Sayed et al., 2023), or applying similarity measurements or linear downscaling techniques (e.g., t-SNE or Kernel Principal Component Analysis) for data conversion (Sharma et al., 2019), as well as the use of feature assignment to pixel positions to convert tabular data into images (Zhu et al., 2021). However, due to the small number of features in our data, such manipulations have been shown to cause overfitting in our experiments.

In this context, we propose to use an image generator based on the combination of convolution and multilayer perceptron, inspired by Deep Generative Adversarial Networks (DCGANs) (Radford et al., 2015). This allows the model to automatically generate 2D images according to the performance of classification.

The generator typically consists of several convolutional and up-sampling operations layers, gradually transforming the 1D sample (quantile-transformed LOS/NLOS data) into a 2D image. As shown in Fig. 9, start with a Fully Connected (FC) layer that takes this 1D sample as input, allowing us to make a data augmentation, from a 1D vector to C feature maps of size 7×7 . C is a constant whose value depends on the number of eigenvalues of the input vector and the size of the data set, in our case, C equals 16. The generated feature maps are then upsampled several times until the optimal image size of $28 \times 28 \times 1$ is achieved. The optimal image size is defined through an ablation

study presented later in Section 5.1. Each layer is associated with a Leaky Rectified Linear Unit (LeakyReLU) activation function which is calculated as follows:

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases} \quad (9)$$

where, α is a small constant representing the slope for negative input, here $\alpha = 0.01$. Compared to the traditional ReLU activation function, LeakyReLU allows a small and non-zero gradient for negative inputs to avoid the issues of “dying neurons” associated with ReLU where gradients become zero and the neuron stops learning.

The output layer uses a hyperbolic tangent activation function, i.e., Tanh, that helps facilitate stable training and ensures that the generated images are within the appropriate intensity range. The tanh function is calculated as follows:

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} \quad (10)$$

where x is the result of a convolution operation between the feature maps $f_{k=1,m}$ of the previous layer and associated filters of weights (kernels):

$$x = \left(\sum_{f_{k=1,m}} \left(\sum_{i,j}^{w,h} w_{i,j} \times x_{i,j} \right) \right) + \text{bias} \quad (11)$$

Here, w and h correspond to the kernel width and height, $w_{i,j}$ is the weight of the kernel at i, j position, $x_{i,j}$ is the value of the feature map at i, j position, and b is the bias. Fig. 10 shows examples of the LOS/NLOS images generated by our proposed image generator from the 6 datasets respectively.

What should be mentioned is that generative AI applied to time-series data presents great challenges in controlling hallucination problems compared to image and text data. In images or text, generative models can often rely on clear, structured patterns — like object contours or linguistic syntax that help maintain coherence in generated outputs. However, time-series data has hardly explicit patterns allowing us to control hallucination. Especially in our case, GNSS features exhibit significant noise and outliers which lead to irregular heavy tail distributions. It is therefore difficult to deal with hallucination problems directly through generated data. To address potential hallucination issues during CarNet's image generation, we adopted an end-to-end verification strategy. That is to say, the quality of the generated images is indirectly controlled by optimizing model accuracy through back-propagation during training. This approach ensures that the generated images accurately support the classification tasks. In this way, the likelihood of hallucinated patterns impacting classification accuracy can be minimized.

4.1.3. GNSS LOS/NLOS classifier

Because our generated images are small in size, it is not appropriate to employ very deep convolutional networks. Instead, it is more advantageous to maximize its feature extraction ability at different scales. To this end, we introduce a residual inception module, which uses several parallel convolutional paths in the classifier.

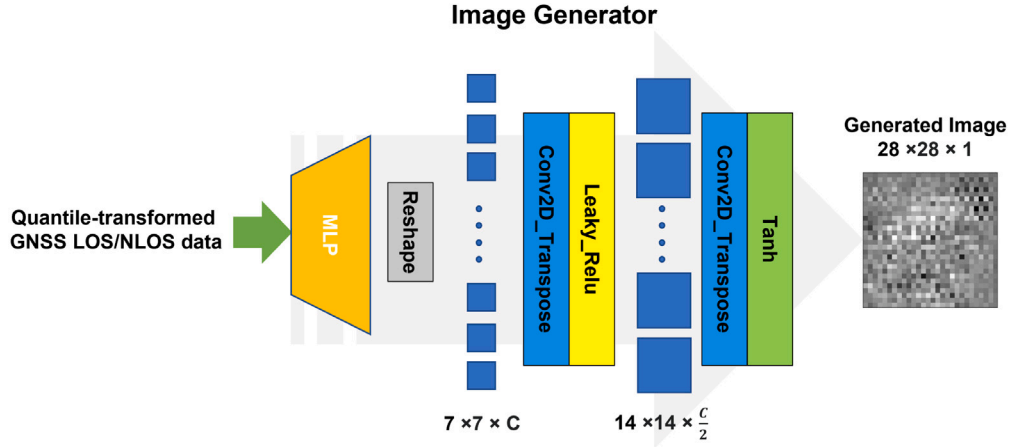
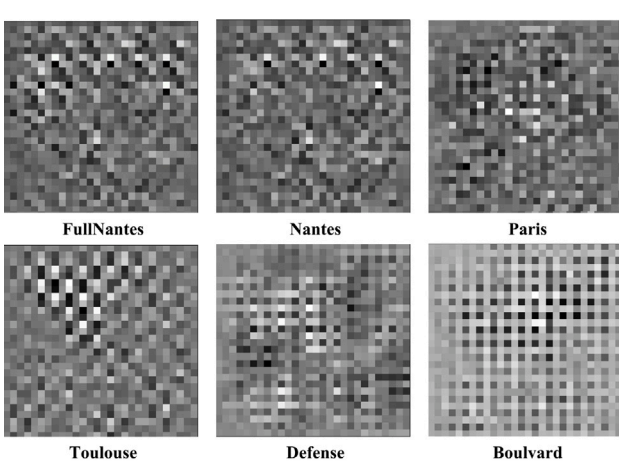
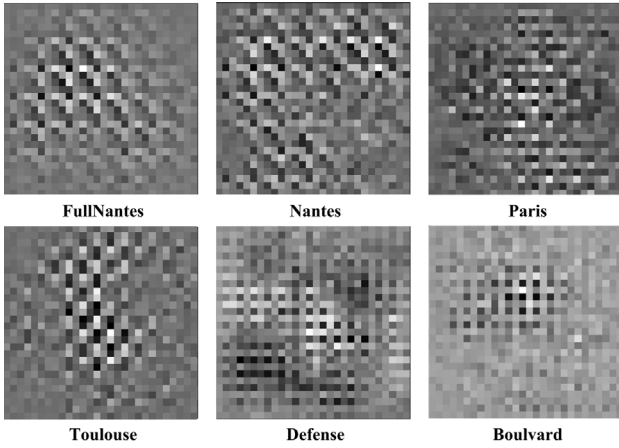


Fig. 9. Design of the proposed Image Generator.



(a) LOS samples



(b) NLOS samples

Fig. 10. Image example of each database for the (a) LOS samples (b) NLOS samples.

As shown in Fig. 11, the input features are first processed by a 1×1 convolutional layer with dimensionality reduction and non-linear transformations which serve to increase the representational power of neural networks while simultaneously reducing computational complexity. We then use three parallel convolutional paths (i.e., path 1, 2 and 3 in Fig. 11) with respectively 1, 2, 3 stacked 3×3 kernels to get

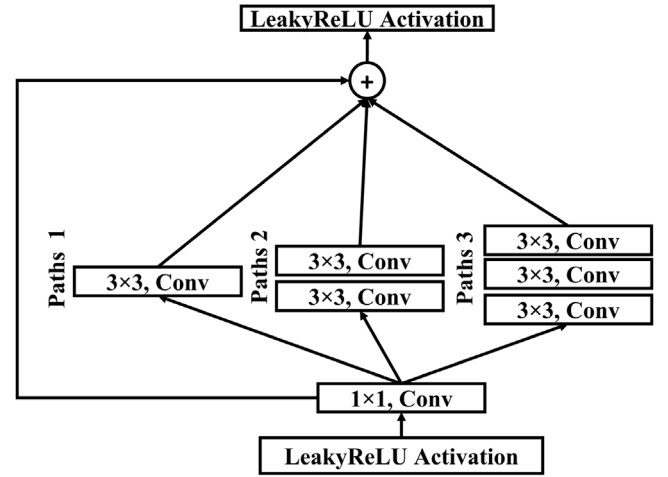


Fig. 11. Proposed inception module for image classifier.

a 3-scale feature extractor module. The three convolutional paths are respectively equivalent to convolutional kernel sizes of 3×3 , 5×5 , and 7×7 . Finally, the residual structure is used to add the output results of each path and the 1×1 convolution preprocessing results as the output of the inception module.

The proposed inception module differs from the existing ones (He et al., 2022) in that at the end of the initial module, we use an addition operation rather than a concatenation structure. By using the addition operation, the Inception module allows the network to learn additive combinations of features, which can be beneficial for capturing complex patterns in the data. This design choice was motivated by the necessity to enhance information flow and gradient propagation through the network.

Similar to our image generator, each convolutional layer in the inception module is associated with a LeakyReLU activation function described in Eq. (9). As illustrated in Fig. 8, the inception step is followed by a maximum pooling function, which allows for the size of each feature map to be reduced by half while maintaining their most significant features. For this purpose, a 2×2 kernel with a stride of 2 is applied over the feature map, and the maximum value is outputted each time. The last inception module is followed by a Global Average Pooling (GAP) function which is calculated for each feature map $f_{k=1,16}$ as follows:

$$GAP(x) = \frac{1}{7 \times 7} \sum_{i,j} x_{i,j} \quad (12)$$

The last layer of the whole network is a fully linked layer containing one neuron, corresponding to a sigmoid function:

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (13)$$

The sigmoid allows us to map the values computed by the neural network into the interval $[0, 1]$, which corresponds to the prediction probability (p). The whole architecture, including the image generator and the image classifier, is trained to reduce the classification Binary Cross Entropy (BCE) loss without conducting intermediate evaluations on image quality due to the absence of the reference images. This means that the image generator aims to create images that can effectively aid the classifier in distinguishing LOS and NLOS based on the input 1D data. The BCE loss is calculated based on the output layer of a batch of images as follows:

$$\text{BCE Loss} = \frac{1}{N} \sum_{i=0}^N -(y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \quad (14)$$

where p_i is the probability of class 1 (NLOS), and $(1 - p_i)$ is the probability of class 0 (LOS). y_i is the binary label (0 or 1). N corresponds to the batch size.

To minimize the BCE loss, the weights of the architecture were updated based on the Stochastic Gradient Descent (SGD) optimizer (Choi et al., 2019). Based on SGD technique, the weight w is updated as described in Algorithm 2.

Algorithm 2 Stochastic gradient descent(SGD) update

Require: Learning rate $\eta = 0.01$

Require: Initial weights w

while Stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{x^{(0)}, x^{(1)}, \dots, x^{(m)}\}$ with corresponding labels $y^{(i)}$.

 Compute gradient estimate:

$$\hat{g} \leftarrow + \frac{1}{m} \nabla_w \sum_{i=0}^m L(f(x^{(i)}, w), y^{(i)})$$

 Apply update: $w \leftarrow w - \eta \hat{g}$

end while

4.2. Cross-validation and evaluation metrics

To enhance the robustness and generalization capabilities of the model, k-fold cross-validation is employed in our experiments among the 6 datasets. Specifically in our case, the model is trained and evaluated 6 times: each time, one dataset is extracted as the test set and the other five datasets as the training set, respectively. The metrics such as sensitivity, specificity and accuracy are evaluated. We define the True Positive (TP) as the number of NLOS samples identified by the classifier as NLOS, True Negative (TN) as the number of LOS samples identified by the classifier as LOS, False Positive (FP) as the number of LOS samples classified as NLOS, False Negative (FN) as the number of NLOS samples classified as LOS. The metrics are calculated as follows:

- Sensitivity: the capacity of the classifier to identify NLOS samples, we denote as P_{NLOS} .

$$P_{NLOS} = \frac{TP}{TP + FN} \quad (15)$$

- Specificity: the capacity of the classifier to identify LOS samples, we denote as P_{LOS} .

$$P_{LOS} = \frac{TN}{TN + FP} \quad (16)$$

- Accuracy: the ratio of correct predictions.

$$\text{Acc} = \frac{TP + TN}{TP + FP + TN + FN} \quad (17)$$

Finally, we average the results of $k(k = 6)$ training to get the final result mP_{NLOS} , mP_{LOS} and $mAcc$

Table 3

Classification cross-validation mean accuracy ($mAcc$) on test set of our GCNN model under different data preprocessing methods with 24-feature set.

Transformation type	Transformation target	Test $mAcc$ (%)
No transformation	None	78.00
Min–Max scaling	$[0, 1]$	77.32
Z-score normalization	Mean 0, Std 1	78.21
Quantile transformation	Normal distribution	81.01
Quantile transformation	Uniform distribution	80.30

5. Experimentation and performance evaluations

5.1. Ablation study

In order to thoroughly evaluate the proposed GCNN model architecture, we conducted a comprehensive ablation study motivated by the need to understand how various factors influence the model performance. Specifically, we examined the following critical aspects: the significance of quantile transformation in enhancing data representation, the impact of feature selection by comparing different numbers of features, the effect of pixel dimension of the generated image on classification accuracy, the choice of optimizer for training efficacy, and the roles of dropout and batch normalization in improving generalization. Additionally, we assessed the influence of the initial learning rate on convergence dynamics. This systematic approach enables us to discern the contributions of each element to the overall effectiveness of our model.

5.1.1. Significance of quantile transformation on model performance

To demonstrate the significance of QT, we trained our model using five different data types: the original dataset, data subjected to Min–Max scaling, data normalized using Z-score normalization, and data transformed with QT to both normal and uniform distributions. The cross-validation mean accuracy ($mAcc$) of our GCNN model for each preprocessing method is summarized in Table 3.

The baseline accuracy without any transformation was 78%. The results highlight the crucial role of data preprocessing in enhancing model performance. Specifically, applying Min–Max scaling resulted in a decrease in accuracy to 77.32%, indicating that this method may not be suitable for our dataset. Z-score normalization yielded a slight improvement, raising the accuracy to 78.21%. In contrast, quantile transformation to a normal distribution achieved the highest accuracy of 81.01%, demonstrating QT's effectiveness in addressing issues related to skewed data and outliers. Similarly, quantile transformation to a uniform distribution also performed well, achieving an accuracy of 80.30%.

These findings underscore the importance of selecting appropriate data preprocessing techniques, with quantile transformation proving particularly advantageous for optimizing the performance of our GCNN model.

5.1.2. Number of features

Based on our previous work, we confirmed that the tree-based models achieve better results for data with the 10 final selected features when the *FullNantes* dataset is used as test set (Zhu et al., 2023a). To further investigate the performance of the proposed model on alternate feature sets, we evaluated its average cross-validation accuracy using the following feature sets:

- (i) **Basic-3:** A 3-feature set including the most basic features from raw measurements – Pseudorange, SNR, Satellite elevation;
- (ii) **State-of-The-Art (SoTA)-3:** A 3-feature set including the essential features typically used in prior studies by Xu et al. (2024), Sun et al. (2020) – Pseudorange residual, SNR, Satellite elevation;

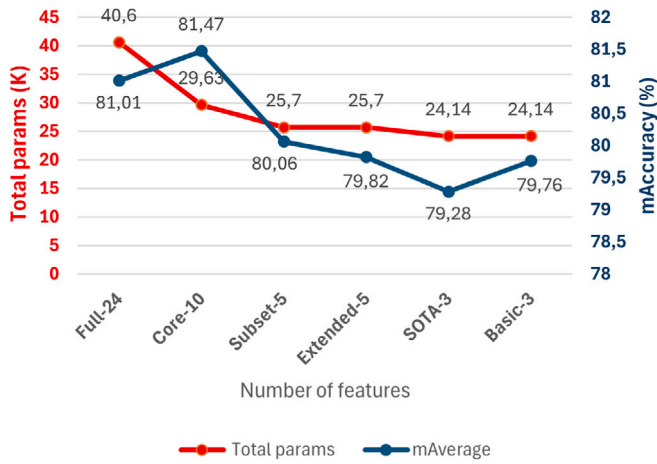


Fig. 12. Effect of feature number on total parameters and mean accuracy ($mAcc$) on test set.

- (iii) **Extended-5:** A 5-feature set expanding the Basic-3 feature set by adding other raw measurements – Pseudorange, SNR, Satellite elevation, Carrier phase, Doppler;
- (iv) **Subset-5:** A 5-feature set consisting of one subset of the Core-10 feature set – SNR, NPR, CMC, LT, number of satellite;
- (v) **Core-10:** A 10-feature set consisting of the core features identified in our previous work, as listed in Section 3.2 – SNR, ϕ , NPR, CMC, CPRRC, LT, number of satellite, Var(SNR), Var(P), Var(Pr);
- (vi) **Full-24:** A 24-feature set including all the candidate features outlined in Section 3.2.

Fig. 12 illustrates the average classification accuracy under cross-validation ($mAcc$) and the total parameters of CarNet across different feature sets. The results demonstrate that the model achieves its highest performance with an $mAcc$ of 81.47% when utilizing the Core-10 feature set, with a total parameter count of 29.63 K. In comparison, the Full-24 feature set achieves a similar accuracy of 81.01% but requires the highest parameter count of 40.6 K, indicating a significantly higher computational cost.

When reducing the features to Subset-5, the accuracy slightly declines to 80.06%, accompanied by a reduction in parameter count to 25.7 K. Similarly, the Extended-5 feature set maintains the same parameter count but results in a lower accuracy of 79.82%. A further reduction to the SoTA-3 feature set leads to an accuracy of 79.28% with 24.14 K parameters, while the Basic-3 feature set achieves a slightly higher accuracy of 79.76% with the same parameter count. It is important to highlight that a 1% decrease in accuracy corresponds to approximately 15600 misclassified samples in the dataset, emphasizing the substantial impact of even minor variations in performance.

Overall, the total parameter count consistently decreases as the number of features is reduced. Considering the trade-off between classification accuracy and computational efficiency, the Core-10 feature set was identified as the optimal configuration for our CarNet model. However, for applications with stringent computational resource constraints, the Subset-5 feature set could also be considered. The remainder of the article will focus on the CarNet model using the Core-10 feature set.

5.1.3. Pixel dimension of the generated image

With the number of features determined, we investigate the impact of image pixel dimension on cross-validation accuracy and computation time, measured as the average time required to train each batch with a batch size of 256. Our model is trained in Google Colab's GPU runtime environment, which provides an Intel Xeon CPU at 2.20 GHz, 13 GB of RAM, a Tesla K80 GPU, and 12 GB of GDDR5 VRAM. Fig. 13 plots

Table 4

Classification cross-validation mean accuracy ($mAcc$) on test set of the proposed CarNet using different optimizers.

Optimizers	mAcc (%)
SGD	81.47
Adam	77.80
RMSprop	80.36
AdamW	80.24
Adadelta	80.14
Adamax	79.62
Adagrad	80.64
Adafactor	79.01
Nadam	76.82
Ftrl	73.49

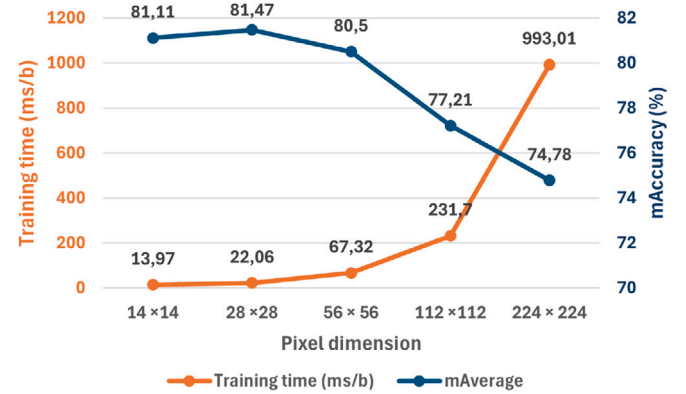


Fig. 13. Effect of generated image pixel dimension on training time and cross-validation mean accuracy ($mAcc$) on test set.

the curves of the cross-validation accuracy and the training time as a function of the generated image size. The best trade-off between accuracy and training time is obtained on the image resolution of 28×28 .

5.1.4. Different optimizers

A benchmarking analysis of 10 commonly used optimizers on the model performance is conducted, i.e., Stochastic Gradient Descent (SGD), Adam, Root Mean Square Propagation (RMSprop), Adamax, Adadelta, Nadam (Choi et al., 2019), Adam with Weight decay (AdamW) (Loshchilov and Hutter, 2017), Adagrad (Duchi et al., 2011), Adafactor (Shazeer and Stern, 2018), Ftrl (McMahan et al., 2013). Table 4 lists the average classification accuracy obtained under cross-validation. It can be concluded that our model achieves the best performance with 81.47% mean accuracy with cross-validation when using the SGD optimizer.

5.1.5. Effect of dropout and batch normalization on model performance

In deep learning, preventing overfitting while achieving high model performance is a key challenge, especially when handling complex datasets. Batch Normalization (BN) and dropout are popular techniques to improve model generalization and convergence. BN normalizes layer inputs, stabilizing and accelerating training, while dropout randomly deactivates neurons during training, preventing overfitting by reducing co-adaptation of neurons.

To explore the effectiveness of these techniques, we evaluated the model using the best optimizer (SGD) under the following conditions:

- Without BN and dropout: The baseline model without any regularization.
- With BN and dropout: BN applied after each convolutional layer and dropout used for regularization.

Table 5
Comparison of Model Performance with different BN and dropout Configurations.

Model Configuration	Training $mAcc$ (%)	Test $mAcc$ (%)	Observed overfitting
Without BN and dropout	84.51	77.83	High
With BN and dropout	82.78	80.56	Low
With only BN	84.11	78.32	Moderate
With only dropout	83.05	81.47	Low

– With only BN and only dropout:

- * With only BN: BN applied after each convolutional layer.
- * With only dropout: dropout applied, without BN.

Table 5 presents the results for each configuration, including the mean accuracy ($mAcc$) for both training and test, the degree of observed overfitting by evaluating the accuracy gap which is the difference between the training and test accuracy:

$$AccuracyGap = Training_mAcc - Test_mAcc \quad (18)$$

There are not any universal thresholds to define the degree of overfitting since it varies depending on the application. In our case, the following criteria are defined:

- If the $AccuracyGap < 3\%$, the observed overfitting is low;
- If the $3\% \leq AccuracyGap < 5\%$, the observed overfitting is moderate;
- If the $5\% \leq AccuracyGap < 10\%$, the observed overfitting is high;
- If the $AccuracyGap \geq 10\%$, the observed overfitting is very high;

The experimental results suggest that dropout alone achieves the best balance between training mean accuracy and test mean accuracy, effectively mitigating overfitting and providing superior generalization to unseen data. While BN helps stabilize the training process, dropout proves to be more effective in preventing overfitting in this case.

5.1.6. Impact of initial learning rate on model performance

In this section, we investigate the effect of the initial learning rate on the performance of our model. The learning rate is a critical hyperparameter that significantly influences the training dynamics and convergence behavior of deep learning models. An optimal initial learning rate is essential for achieving fast convergence and avoiding suboptimal solutions.

To optimize this aspect, we implemented a dynamic learning rate strategy in our experiments. The model starts training with an initial learning rate, and if there is no improvement in validation performance after every 50 epochs, the learning rate is reduced by a factor of 0.1. This approach allows for rapid learning in the initial phases while refining the learning process as training progresses, thereby accelerating the model's convergence and helping to avoid local optimum solutions (Loshchilov and Hutter, 2016; Smith, 2017).

In this investigation, we tested the following initial learning rates: 1×10^{-4} , 1×10^{-3} , 1×10^{-2} , and 1×10^{-1} . Each configuration was trained under consistent conditions, incorporating dropout for regularization. We monitored the following metrics: mean accuracy ($mAcc$) for both training and test, the degree of overfitting observed, and the convergence speed.

Table 6 presents the results of experiments evaluating the impact of different initial learning rates on our model's performance. The findings emphasize the importance of selecting an appropriate initial

learning rate. Using a learning rate of 1×10^{-2} strikes the best balance between training performance and generalization capability, achieving moderate overfitting and satisfactory test accuracy.

In contrast, while using learning rates of 1×10^{-1} resulted in higher training accuracy, it exhibited significant overfitting, highlighting the need to monitor validation performance alongside training accuracy. Using a learning rate of 1×10^{-3} and 1×10^{-4} , although associated with lower training and test accuracy compared to other configurations, demonstrated low overfitting. This indicates that, despite struggling to learn effectively, the model generalized better than at higher learning rates. This presents a trade-off: while a very low learning rate can help prevent overfitting, it may also hinder the model's ability to capture essential patterns in the data and slow down its convergence speed.

5.1.7. Impact of batch size on model performance

Batch size significantly affects both training efficiency and model performance. In this study, we evaluated batch sizes of 512, 256, 128, and 64 to understand their impact on cross-validation accuracy.

Table 7 shows that a batch size of 256 yields the highest mean accuracy ($mAcc$) at 81.47%, suggesting an optimal balance between training stability and generalization. Larger batch sizes (512) result in slightly lower accuracy, possibly due to less frequent updates. In contrast, smaller batch sizes of 128 and 64 reduce accuracy, as it may struggle to capture the necessary patterns due to the vehicular dynamic. Thus, a batch size of 256 offers the best compromise between accuracy and training efficiency.

5.2. Performance comparative study

5.2.1. Model prediction performance evaluation and benchmarking

After the ablation study, the architecture as well as the hyperparameters of the CarNet model can be fixed: we will use 10 features to generate images of 28×28 using SGD optimizer. A thorough benchmarking study is conducted by comparing our proposed GCNN-based CarNet model with existing mainstream AI methods: the tree-based ML methods (i.e., Decision Trees, Random Forest, ExtraTrees, Gradient Boosting, XGboost, AdaBoost, Light Gradient Boost Machine – LGBM, Histogram-based Gradient Boosting), Logistic Regression, Naïve Bayes, K-Nearest Neighbors (KNN), the deep learning-based methods (Multiple Layer Perceptron – MLP, 1D-CNN, Multi-head Attention). The hyperparameters of the ML methods are optimized by a grid search. In addition, we also implemented the approach proposed in Sayed et al. (2023), where a simple replication and interpolation algorithm is used to transform 1D time series into 2D images. After that, a pre-trained CNN model is used to classify the generated images. This state-of-the-art approach is denoted as RIG-CNN: a Replication and Interpolation-based image Generator with CNN.

The implemented deep learning network structures for benchmark are listed in Table 8. The structure extraction layer is defined as $((C_{in}, C_{out}), \text{kernel}, \text{stride}, \text{padding})$ for convolutional layers, $(\text{kernel}, \text{stride})$ for maxpooling layers, (C_{in}, C_{out}) for GAP and FC layers, $(\text{input size}, \text{hidden size}, \text{num of layers})$ for LSTM layers, $(C_{in}, \text{nums_head})$ for self-attention.

Table 9 summarizes the classification performance with cross-validation of our proposed model against mainstream ML techniques. Our proposed GCNN-based CarNet model can achieve the best trade-off between mP_{LOS} and mP_{NLOS} , i.e., 83.3% and 70.99% respectively. The overall $mAcc$ of the proposed CarNet model reaches 81.47%. The second-best performance is achieved by Gradient Boosting (GB), with an average accuracy of 80.39%. Note that the total data size is approximately 1.56 million so even a 1% difference accounts for 15600 samples. What should be highlighted is that during the cross-validation, each test dataset remains entirely unseen by the training procedure, which shows the CarNet model's high generalizability and robustness.

In addition to the mainstream AI models, we compared our proposed CarNet model with two state-of-the-art approaches (Xu et al.,

Table 6
Effect of initial learning rate on the performance of our proposed CarNet model.

Initial learning rate	Training $mAcc$ (%)	Test $mAcc$ (%)	Overfitting observed	Convergence speed
1×10^{-4}	81.74	80.55	Low	Very slow
1×10^{-3}	81.89	80.48	Low	Moderate
1×10^{-2}	83.05	81.47	Low	Fast
1×10^{-1}	87.62	76.51	High	Very fast

Table 7

Classification cross-validation mean accuracy ($mAcc$) on test set of our CarNet model using different batch sizes during training.

Batch size	512	256	128	64
Test $mAcc$ (%)	80.17	81.47	80.48	79.69

Table 8

Structure for all the comparative deep neural networks.

Method	Extraction layer architecture
MLP	(10, 150) FC
CNN1D	(10, 150) FC ((10, 1), (10, 8), 3, 1, same) Conv (2, 2) Maxpooling ((5, 8), (5, 16), 3, 1, same) Conv ((5, 16), 16) GAP
CNN+LSTM	((10, 1), (10, 8), 3, 1, same) Conv (2, 2) Maxpooling ((5, 8), (5, 16), 3, 1, same) Conv (16, 32, 1) LSTM
Multi-head Attention	(10, 2) Self_attention BatchNorm

Table 9

Benchmarking classification cross-validation performance of our proposed CarNet model against mainstream AI techniques.

Method	mP_{LOS} (%)	mP_{NLOS} (%)	$mAcc$ (%)
ExtraTrees	81.69	66.79	79.18
Decision Tree	80.88	51.16	70.23
RandomForest	83.56	64.55	78.21
XGboost	82.29	65.05	78.00
Gradient Boosting	83.20	70.04	80.39
AdaBoost	83.23	65.43	79.07
LGBM	83.55	67.79	79.42
HistGradient Boosting	83.02	68.97	80.00
Logistic Regression	80.95	68.11	77.97
Naïve Bayes	83.05	61.71	76.65
KNN	80.67	59.17	74.92
MLP	81.70	68.97	78.65
RIG-CNN (Sayed et al., 2023)	81.80	69.18	79.23
CNN1D	81.24	69.84	78.65
CNN+LSTM	80.82	70.40	78.84
Multi-head Attention	80.61	69.08	78.20
CarNet	83.30	70.99	81.47

Table 10

Comparison of the proposed CarNet model with two state-of-the-art models in terms of cross validation mP_{LOS} (%), mP_{NLOS} (%), and $mAcc$ (%)

Methods	mP_{LOS} (%)	mP_{NLOS} (%)	$mAcc$ (%)
FCNN (Xu et al., 2024)	80.41	37.7	42.29
GBDT (Sun et al., 2020)	83.09	56.94	72.3
CarNet	83.30	70.99	81.47

2024; Sun et al., 2020), which utilized respectively Fully Connected Neural Network (FCNN) and Gradient Boosting Decision Tree (GBDT) models to classify GNSS LOS and NLOS receptions. Support Vector Machine (SVM) is also a commonly used classifier for GNSS LOS/NLOS signal classification. However, its high computational cost makes it impractical for large-scale datasets (Graf et al., 2004; Wu and Chang, 2005; Lee and Mangasarian, 2001). Therefore, SVM-based methods were excluded from our benchmarks.

Table 11

Inference latency for one sample in milliseconds of our model compared with the top 2 models in terms of $mAcc$ from Table 9.

Model	Gradient Boosting	HistGradient Boosting	GCNN (CPU)	GCNN (GPU)
Latency (ms)	0.48	1.43	4.68	2.91

Table 10 summarizes the performance of CarNet model against the two state-of-the-art approaches. The implementation of FCNN and GBDT strictly follows the descriptions in the respective articles, using three features (SNR, Satellite elevation, and pseudorange residual, which constitute the SoTA-3 feature set described in Section 5.1.2), with Min-Max normalization applied for data preprocessing. In contrast, our proposed CarNet model utilizes QT preprocessing and 10 core features, as outlined earlier. The architecture of the FCNN which achieved the best performance in Xu et al. (2024) is implemented here for comparison.

The results demonstrate that CarNet, leveraging the core-10 feature set, significantly outperforms the other methods across all cross-validation metrics, achieving a 14.05% improvement in mP_{NLOS} and a 9.17% improvement in $mAcc$. The FCNN and GBDT encounter significant difficulties in accurately classifying the NLOS signals across various urban scenarios in different cities. These findings highlight the robustness and generalization ability of CarNet in complex scenarios, showing that incorporating more features and using an advanced model can greatly enhance NLOS classification and overall accuracy.

5.2.2. Inference latency evaluation

To assess the model's potential for real-time deployment, we conducted predictions on six distinct datasets, processing each sample individually with a batch size of one for CarNet. The evaluation was conducted on Google Colab, with configuration details provided in Section 5.1.3. For each dataset, we recorded the total computation time for all predictions and calculated the average latency by dividing this cumulative time by the number of samples processed. Subsequently, the average latency values from each dataset were combined to produce an overall latency measure, providing a clear view of the model's responsiveness and efficiency.

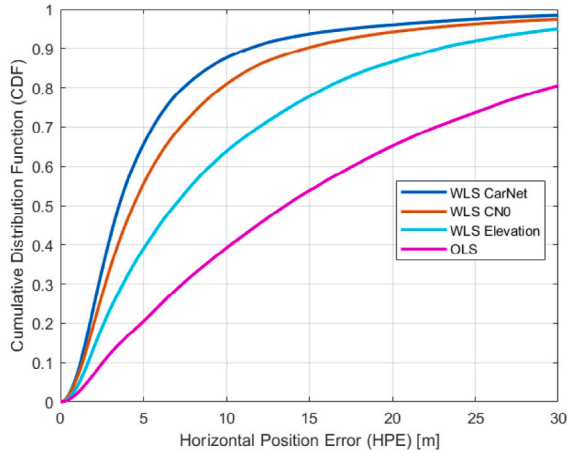
Based on the $mAcc$ metric, we selected the top three models (including the proposed one) from Table 9 for a focused comparison of latency. Table 11 illustrates that while Gradient Boosting achieves the lowest sample inference latency at 0.48 ms, both the GCNN models maintain response times of 4.68 ms on the CPU and 2.91 ms on the GPU. The measurement frequency of a typical GNSS receiver is 5 Hz, allowing the model to infer approximately 42 samples using only CPU and around 68 samples with GPU during a single data period. The number of samples that the CarNet is capable of handling at each sample period is much higher than the average number of visible satellites (around 25) at a single epoch in full GNSS constellations. This demonstrates the potential for utilizing CarNet in real-time applications.

5.2.3. Positioning performance evaluation

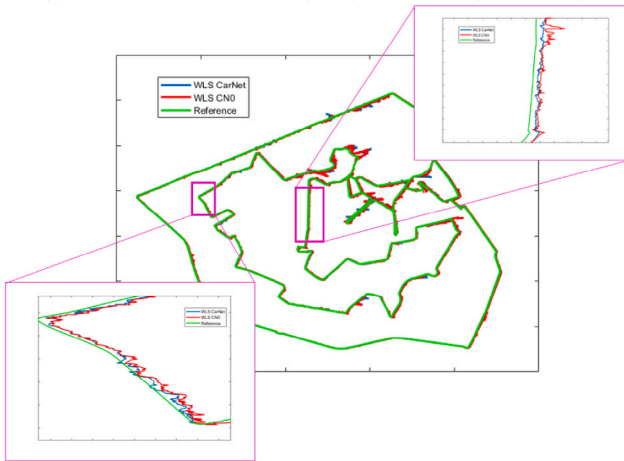
The ultimate objective of training the GNSS LOS/NLOS classifier is to contribute to improving the GNSS positioning accuracy in challenging environments. To evaluate the positioning accuracy gain using LOS/NLOS information predicted by the CarNet, we implement a simple reweighting strategy in the WLS position estimator according to the

Table 12
Horizontal Position Error (HPE) of all the dataset.

Methods \ HPE [m]	Medium (50%)	75%	95%
OLS	13.58	25.87	56.52
WLS Elevation	6.88	13.86	30.12
WLS CNO	4.41	8.37	20.53
WLS CarNet	3.54	6.34	17.50



(a) Horizontal Position Error (HPE) comparison (all datasets)



(b) Trajectories comparison (an example of Nantes Center dataset)

Fig. 14. Analysis on positioning error and trajectory.

hybrid approach proposed in Zhu (2018), where the weight assigned to NLOS is decreased by a constant multiplier which equals to 10 in this paper. That means the measurements from the LOS satellites identified by CarNet are assigned ten times higher weight than the NLOS satellites. Three other state-of-the-art positioning estimators are also implemented to make the comparison: Ordinary Least Square (OLS) as baseline algorithm, WLS according to C/N_0 as well as WLS with satellite elevation.

Fig. 14(a) plots the Cumulative Distribution Function (CDF) of the Horizontal Position Error (HPE) of all the datasets using different position estimators mentioned previously and Table 12 summarizes the corresponding statistics. Compared to the traditional weighting methods using C/N_0 or satellite elevation, the WLS method using CarNet prediction has the highest accuracy whose medium HPE achieves 3.54 m and 75% accuracy of 6.34 m based on the 9 hours' data collected in different challenging environments. Fig. 14(b) shows an example of the estimated trajectory for the Full Nantes dataset, where the

WLS CarNet and WLS C/N_0 are compared together with the reference trajectory. We can observe fewer huge errors appearing in the estimated trajectory by WLS CarNet, which means the faulty measurements are effectively attenuated by the LOS/NLOS information provided by CarNet.

6. Conclusions and perspectives

This paper proposes a robust GNSS LOS/NLOS classifier, CarNet, by converting multivariate time-series data into images using Generative CNN (GCNN). Then a multi-scale inception-based classifier is proposed to classify the generated image. The main advantages of the proposed architecture are that it can first realize feature augmentation with an image generator then the classifier can extract higher-level hierarchical features compared to the traditional ML method. The proposed architecture is trained and tested using real vehicle data collected in six distinct challenging environments over 9 h. The performances of the CarNet are evaluated using cross-validation over different cities on unseen data. Its classification accuracy, i.e., 81.47%, is proven to be the highest compared to the existing mainstream AI methods while maintaining an optimal trade-off between LOS and NLOS precision. The positioning accuracy can also be improved by taking the LOS/NLOS information provided by CarNet to adjust the satellite measurement weights. The 75% HPE is reduced by CarNet from 8.37 m and 13.86 m obtained through C/N_0 and satellite elevation weighting respectively, down to 6.34 m. The evaluation results of the average inference latency, i.e., 4.68 ms and 2.91 ms per sample on CPU and GPU respectively, show a strong potential of embedding CarNet for real-time applications.

One future work would be merging environmental multimodal information with the generated feature image since the architecture of the CarNet facilitates the fusion with other data types such as image or even semantic information. Moreover, using the current pre-trained vehicular model, transfer learning techniques can be applied to adapt the model for other modes of transportation, such as bikes, e-scooters, or even pedestrians.

CRedit authorship contribution statement

Ni Zhu: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Ruiwen He:** Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis. **Zhiqiang Wang:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported partly by the French National Research Agency (ANR) in the framework of the project ReSilientGAIA ANR-23-CE22-0004-01.

Data availability

The authors do not have permission to share data.

References

- Abdel-Basset, M., Hawash, H., Chang, V., Chakraborty, R.K., Ryan, M., 2020. Deep learning for heterogeneous human activity recognition in complex IoT applications. *IEEE Internet Things J.* 9 (8), 5653–5665.
- Baldini, G., Giuliani, R., Dimc, F., 2019. Physical layer authentication of internet of things wireless devices using convolutional neural networks and recurrence plots. *Internet Technol. Lett.* Volume 2 (2), e81. <http://dx.doi.org/10.1002/itl2.81>.
- Betaille, D., Peyret, F., Ortiz, M., Miquel, S., Fontenay, L., 2013. A new modeling based on urban trenches to improve GNSS positioning quality of service in cities. *IEEE Intell. Transp. Syst. Mag.* 5 (3), 59–70.
- Bishop, C.M., Nasrabadi, N.M., 2006. *Pattern Recognition and Machine Learning*, vol. 4, (4), Springer.
- Blanch, J., Walter, T., Enge, P., Lee, Y., Pervan, B., Rippl, M., Spletter, A., 2012. Advanced RAIM user algorithm description: Integrity support message processing, fault detection, exclusion, and protection level calculation. In: *Proceedings of the 25th International Technical Meeting of the Satellite Division of the Institute of Navigation. ION GNSS 2012*, pp. 2828–2849.
- Box, G.E., Cox, D.R., 1964. An analysis of transformations. *J. R. Stat. Soc. Ser. B Stat. Methodol.* 26 (2), 211–243.
- Chen, J., Wang, X., 2022. Non-intrusive load monitoring using gramian angular field color encoding in edge computing. *Chin. J. Electron.* 31 (4), 595–603.
- Choi, D., Shallue, C.J., Nado, Z., Lee, J., Maddison, C.J., Dahl, G.E., 2019. On empirical comparisons of optimizers for deep learning. *arXiv preprint arXiv:1910.05446*.
- Duchi, J., Hazan, E., Singer, Y., 2011. Adaptive subgradient methods for online learning and stochastic optimization. *J. Mach. Learn. Res.* 12 (7).
- Eckmann, J.P., Kamphorst, S.O., Ruelle, D., et al., 1995. Recurrence plots of dynamical systems. *World Sci. Ser. Nonlinear Sci. Ser. A* 16, 441–446.
- Fohlmeister, F., Iliopoulos, A., Sgammmini, M., Antreich, F., Nossek, J.A., 2017. Dual polarization beamforming algorithm for multipath mitigation in GNSS. *Signal Process.* 138, 86–97.
- Friedman, J., Hastie, T., Tibshirani, R., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
- Graf, H., Cosatto, E., Bottou, L., Dourdanovic, I., Vapnik, V., 2004. Parallel support vector machines: The cascade SVM. *Adv. Neural Inf. Process. Syst.* 17.
- Groves, P.D., Jiang, Z., Skelton, B., Cross, P.A., Lau, L., Adane, Y., Kale, I., 2010. Novel multipath mitigation methods using a dual-polarization antenna. In: *Proceedings of the 23rd International Technical Meeting of the Satellite Division of the Institute of Navigation. ION GNSS 2010*, pp. 140–151.
- He, R., Benhabiles, H., Windal, F., Even, G., Audebert, C., Decherf, A., Collard, D., Taleb-Ahmed, A., 2022. A CNN-based methodology for cow heat analysis from endoscopic images. *Appl. Intell.* 52 (8), 8372–8385.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778.
- Hsu, L.T., 2017. GNSS multipath detection using a machine learning approach. In: *2017 IEEE 20th International Conference on Intelligent Transportation Systems. ITSC, IEEE*, pp. 1–6.
- Huang, N.E., Shen, Z., Long, S.R., Wu, M.C., Shih, H.H., Zheng, Q., Yen, N.C., Tung, C.C., Liu, H.H., 1998. The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis. *Proc. R. Soc. Lond. Ser. A Math. Phys. Eng. Sci.* 454 (1971), 903–995.
- Izenman, A.J., 2008. *Modern Multivariate Statistical Techniques*, vol. 1, Springer.
- Jiang, Z., Groves, P.D., 2014. NLOS GPS signal detection using a dual-polarisation antenna. *GPS Solutions* 18, 15–26.
- Lee, Y.J., Mangasarian, O.L., 2001. RSVM: Reduced support vector machines. In: *Proceedings of the 2001 SIAM International Conference on Data Mining. SIAM*, pp. 1–17.
- Leick, A., Rapoport, L., Tatarnikov, D., 2015. *GPS Satellite Surveying*. John Wiley & Sons.
- Li, S., Ji, J., Xu, Y., Sun, X., Feng, K., Sun, B., Wang, Y., Gu, F., Zhang, K., Ni, Q., 2023. IFD-MDCN: Multibranch denoising convolutional networks with improved flow direction strategy for intelligent fault diagnosis of rolling bearings under noisy conditions. *Reliab. Eng. Syst. Saf.* 237, 109387.
- Loshchilov, I., Hutter, F., 2016. SGDR: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*.
- Loshchilov, I., Hutter, F., 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.
- Luo, H., Liu, J., He, D., 2023. GNSS signal labeling, classification and positioning in urban scenes based on PSO-LGBM-WLS algorithm. *IEEE Trans. Instrum. Meas.*
- Marais, J., Kazim, S.A., Cocheril, Y., Meurie, C., 2020. Multipath and NLOS detection based on the combination of CNO values and a fish-eye camera. In: *2020 European Navigation Conference. ENC, IEEE*, pp. 1–13.
- McMahan, H.B., Holt, G., Sculley, D., Young, M., Ebner, D., Grady, J., Nie, L., Phillips, T., Davydov, E., Golovin, D., et al., 2013. Ad click prediction: a view from the trenches. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1222–1230.
- Miura, S., Hsu, L.T., Chen, F., Kamijo, S., 2015. GPS error correction with pseudorange evaluation using three-dimensional maps. *IEEE Trans. Intell. Transp. Syst.* 16 (6), 3104–3115.
- Ng, H.F., Zhang, G., Hsu, L.T., 2019. Range-based 3D mapping aided GNSS with NLOS correction based on skyplot with building boundaries. In: *Proceedings of the ION 2019 Pacific PNT Meeting*, pp. 737–751.
- Ozeki, T., Kubo, N., 2022. Gns nlos signal classification based on machine learning and pseudorange residual check. *Front. Robot. AI* 9, 868608.
- Radford, A., Metz, L., Chintala, S., 2015. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*.
- Rousseeuw, P.J., Leroy, A.M., 2005. *Robust Regression and Outlier Detection*. Wiley.
- Sayed, A.N., Himeur, Y., Bensaali, F., 2023. From time-series to 2d images for building occupancy prediction using deep transfer learning. *Eng. Appl. Artif. Intell.* 119, 105786.
- Sharma, A., Vans, E., Shigemizu, D., Boroevich, K.A., Tsunoda, T., 2019. DeepInsight: A methodology to transform a non-image data to an image for convolution neural network architecture. *Sci. Rep.* 9 (1), 11399.
- Shazeer, N., Stern, M., 2018. Adafactor: Adaptive learning rates with sublinear memory cost. In: *International Conference on Machine Learning. PMLR*, pp. 4596–4604.
- Smith, L.N., 2017. Cyclical learning rates for training neural networks. In: *2017 IEEE Winter Conference on Applications of Computer Vision. WACV, IEEE*, pp. 464–472.
- Song, X., Peng, Z., Song, S., Stojanovic, V., 2024. Anti-disturbance state estimation for PDT-switched RDNNs utilizing time-sampling and space-splitting measurements. *Commun. Nonlinear Sci. Numer. Simul.* 132, 107945.
- Sun, R., Wang, G., Zhang, W., Hsu, L.T., Ochieng, W.Y., 2020. A gradient boosting decision tree based GPS signal reception classification algorithm. *Appl. Soft Comput.* 86, 105942. <http://dx.doi.org/10.1016/j.asoc.2019.105942>.
- Suzuki, T., Amano, Y., 2021. NLOS multipath classification of GNSS signal correlation output using machine learning. *Sensors* 21 (7), 2503.
- Suzuki, T., Kusama, K., Amano, Y., 2020. NLOS multipath detection using convolutional neural network. In: *Proceedings of the 33rd International Technical Meeting of the Satellite Division of the Institute of Navigation. ION GNSS+ 2020*, pp. 2989–3000.
- Szegedy, C., Ioffe, S., Vanhoucke, V., Alemi, A., 2017. Inception-v4, inception-resnet and the impact of residual connections on learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 31, No. 1.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z., 2016. Rethinking the inception architecture for computer vision. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818–2826.
- Tiku, M.L., 1988. Robust statistics with applications. *Statistics* 7 (3), 219–230.
- Tokura, H., Kubo, N., 2017. Efficient satellite selection method for instantaneous RTK-GNSS in challenging environments. *Trans. Japan Soc. Aeronaut. Space Sci.* 60 (4), 221–229.
- Vagle, N., Broumandan, A., Jafarnia-Jahromi, A., Lachapelle, G., 2016. Performance analysis of GNSS multipath mitigation using antenna arrays. *J. Glob. Position. Syst.* 14, 1–15.
- Wang, J., Chen, X., Shi, C., Liu, J., 2023a. Robust M-estimation-based ICKF for GNSS outlier mitigation in GNSS/SINS navigation applications. *IEEE Trans. Instrum. Meas.*
- Wang, Z., Ezukwoke, K., Hoayek, A., Batton-Hubert, M., Boucher, X., 2023b. Natural language processing (NLP) and association rules (AR)-based knowledge extraction for intelligent fault analysis: a case study in semiconductor industry. *J. Intell. Manuf.* 1–16.
- Wang, Z., Oates, T., et al., 2015. Encoding time series as images for visual inspection and classification using tiled convolutional neural networks. In: *Workshops At the Twenty-Ninth AAAI Conference on Artificial Intelligence*, vol. 1, AAAI Menlo Park, CA, USA.
- Wasserman, L., 2013. *All of statistics: A concise course in statistical inference*. Springer Science & Business Media.
- Wu, J., Chang, C.C., 2005. Training data reduction to speed up SVM training. *IEEE Trans. Neural Netw.* 16 (6), 1544–1551. <http://dx.doi.org/10.1109/TNN.2005.855604>.
- Xu, H., Angrisano, A., Gaglione, S., Hsu, L.T., 2020. Machine learning based LOS/NLOS classifier and robust estimator for GNSS shadow matching. *Satell. Navig.* 1, 1–12.
- Xu, Y., Feng, K., Yan, X., Sheng, X., Sun, B., Liu, Z., Yan, R., 2023. Cross-modal fusion convolutional neural networks with online soft-label training strategy for mechanical fault diagnosis. *IEEE Trans. Ind. Inform.* 20 (1), 73–84.
- Xu, P., Zhang, G., Yang, B., Hsu, L.T., 2024. Machine learning in GNSS multipath/NLOS mitigation: Review and benchmark. *IEEE Aerosp. Electron. Syst. Mag.* 39, 26–44. <http://dx.doi.org/10.1109/MAES.2024.3395182>.

- Yang, C.L., Chen, Z.X., Yang, C.Y., 2019. Sensor classification using convolutional neural network by encoding multivariate time series as two-dimensional colored images. *Sensors* 20 (1), 168.
- Yozevitch, R., Moshe, B.B., Weissman, A., 2016. A robust GNSS LOS/NLOS signal classifier. *Navig., J. Inst. Navig.* 63 (4), 429–442.
- Zhang, G., Xu, B., Hsu, L.T., 2018. GNSS shadow matching based on intelligent LOS/NLOS classifier. In: 16th IAIN World Congr, Vol. 2018.
- Zhang, G., Xu, P., Xu, H., Hsu, L.T., 2021. Prediction on the urban GNSS measurement uncertainty based on deep learning networks with long short-term memory. *IEEE Sens. J.* 21 (18), 20563–20577.
- Zhu, N., 2018. GNSS Propagation Channel Modeling in Constrained Environments: Contribution to the Improvement of the Geolocation Service Quality (Ph.D. thesis). Université de Lille.
- Zhu, N., Belemoualem, C., Renaudin, V., 2023a. A portfolio of machine learning-based GNSS LOS/NLOS classification in urban environments. In: Proceedings of the IEEE SENSORS Conference. 29 October 2023 - 01 November, Vienna, Austria, IEEE, pp. 1–4.
- Zhu, N., Betaille, D., Marais, J., Berbineau, M., 2020. GNSS integrity monitoring schemes for terrestrial applications in harsh signal environments. *IEEE Intell. Transp. Syst. Mag.* 12 (3), 81–91.
- Zhu, N., Bouronopoulos, A., Leduc, T., Servièrès, M., Renaudin, V., 2023b. Evaluation of the human body mask effects on GNSS wearable devices for outdoor pedestrian navigation using fisheye sky views. In: 2023 IEEE/ION Position, Location and Navigation Symposium. PLANS, IEEE, pp. 841–850.
- Zhu, Y., Brettin, T., Xia, F., Partin, A., Shukla, M., Yoo, H., Evrard, Y.A., Doroshov, J.H., Stevens, R.L., 2021. Converting tabular data into images for deep learning with convolutional neural networks. *Sci. Rep.* 11 (1), 11325.
- Zhu, N., Marais, J., Betaille, D., Berbineau, M., 2017. Evaluation and comparison of GNSS navigation algorithms including FDE for urban transport applications. In: Proceedings of the 2017 International Technical Meeting of the Institute of Navigation. pp. 51–69.